Jasper: Engineering a Global Workspace for Latent Reasoning in Hybrid Retention Models

Jasper Research Project

August 2026

Abstract

Recent work from Anthropic has revealed that language models develop a compact, privileged internal workspace—the J-SPACE—that causally carries multi-step reasoning, while Samsung's Tiny Recursive Model (TRM) has demonstrated that iterative refinement through a tiny recurrent network can substitute for parameter count on hard reasoning tasks. We present Jasper, an architecture that combines these two insights into a single system: a hybrid retention/attention backbone with an explicitly engineered workspace module and a recurrent core that iterates on the workspace state. The key architectural observation is that the retention layer's fixed-size compressed state is functionally shaped like the J-SPACE—a coincidence that makes the combination more powerful than applying the same ideas to a pure transformer. The retention layer (decayed linear attention) preserves the key property—a fixed-size, input-dependent state—while being simple and fast on the target hardware. The recurrent core combines iteration outputs via learned attention residuals (inspired by Kimi K3), which bound output magnitude and provide stable gradient flow across iterations. We argue that this approach is a direct and more efficient alternative to chain-of-thought "thinking tokens": it reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and allows trading inference-time compute depth for parameter count. We describe a proof-of-concept experiment that trains three parameter-matched model variants on synthetic reasoning tasks with controllable depth, and outline the pre-registered criteria for a go/no-go decision on scaling.

Contents

1 Introduction

Large language models have demonstrated impressive reasoning capabilities, but the dominant paradigm for scaling reasoning—generating long chains of "thinking tokens" before producing an answer—has fundamental inefficiencies. Each thinking token requires a full forward pass through the model, the key-value cache grows linearly with the length of the reasoning trace, and the entire chain must be serialized in token space, consuming both memory and latency. On constrained deployment targets such as web browsers or mobile devices, this memory growth is the binding constraint.

Two recent findings suggest an alternative path. First, Anthropic's discovery of the J-SPACE (Anthropic, 2026) revealed that the reasoning-critical machinery inside a language model is already small: a compact set of verbalizable representations, occupying less than 10% of activation variance, carries the causal bulk of multi-step reasoning. This workspace is not the full residual stream—it is a privileged, broadcasted subset that functions analogously to the global workspace in cognitive science (Baars, 2005). Second, Samsung's Tiny Recursive Model (TRM) (Jolicoeur-Martineau, 2025) and Geiping et al.'s recurrent-depth transformers (Geiping et al., 2025) have shown that iterative refinement through a small recurrent network can substitute for parameter count: a 7M-parameter model recursing on itself can outperform models with 1000× more parameters on reasoning benchmarks.

Jasper combines these insights. We engineer an explicit workspace module—a compact set of learned slot vectors that serve as a persistent scratchpad—and pair it with a recurrent core that iterates on the workspace state. The backbone is a hybrid retention/attention model. The retention layer (decayed linear attention (Sun et al., 2024)) maintains a fixed-size, input-dependent compressed state—the key property that makes it functionally shaped like the J-SPACE. The architecture already has a mechanism that resembles the global workspace, so the engineered workspace and the retention state reinforce each other rather than competing for the same functional role.

The result is a model that reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and can trade inference-time compute (more loop iterations) for parameter count. We argue that this is a direct alternative to thinking tokens—one that is more memory-efficient, faster, and potentially more capable of reasoning that is not easily expressed in words.

2 Background

2.1 The J-Space: A Global Workspace in Language Models

Anthropic (2026) introduced the Jacobian lens (J-lens), an interpretability technique that identifies which concepts a language model is *poised to verbalize* at any point in its processing. Applying the J-lens across layers reveals a set of representations—the J-SPACE—that exhibits the functional properties of a global workspace (Dehaene, 2014; Baars, 2005):

- **Verbal report:** J-space contents can be read out as words the model could say.
- **Directed modulation:** Instructing the model to think about a concept causes that concept to appear in the J-space.
- **Silent reasoning:** Intermediate steps of multi-step problems appear in the J-space before the model verbalizes them.
- **Selective causality:** Ablating the J-space destroys multi-step reasoning while leaving single-hop recall intact.
- **Compactness:** The J-space carries fewer than ∼25 concepts at a time and occupies <10% of activation variance.

Crucially, the J-SPACE was not designed—it emerged during training. But its properties are

exactly what one would want from an engineered reasoning workspace: it is small, privileged, broadcasted, and causally responsible for multi-step reasoning. This raises the question: *can we engineer such a workspace explicitly, and does doing so improve reasoning in a small model?*

2.2 Iterative Reasoning with Tiny Networks

Jolicoeur-Martineau (2025) introduced the Tiny Recursive Model (TRM), a 7M-parameter network that recurses on itself to iteratively refine answers. TRM alternates between a "think" step (updating a latent scratchpad $\mathbf{z}$) and an "act" step (updating a solution embedding $\mathbf{y}$), unrolled for up to 16 iterations with deep supervision. Despite having less than 0.01% of the parameters of frontier LLMs, TRM achieves 45% accuracy on ARC-AGI-1 and 8% on ARC-AGI-2, outperforming models like DeepSeek R1 and o3-mini on these benchmarks.

Independently, Geiping et al. (2025) demonstrated that recurrent-depth transformers—which iterate a transformer block for a variable number of steps at inference time—can scale test-time compute in latent space. Their 3.5B-parameter model, trained on 800B tokens, improves performance on reasoning benchmarks with additional inference-time iterations, sometimes matching the performance of a 50B-parameter model.

Both works share a key insight: *depth (iterations) can substitute for width (parameters)*. But both use attention-based cores, which means the key-value cache grows with sequence length, and the recurrent state is carried in the residual stream—a high-dimensional, unstructured representation.

2.3 Retention: Decayed Linear Attention

Sun et al. (2024) introduced the Retentive Network (RetNet), which replaces standard attention's softmax normalization with a decayed linear attention formulation. The retention layer computes:

$$y_t = \sum_{s \le t} \gamma^{t-s} \cdot (C_t^\top B_s) \cdot V_s \tag{1}$$

where B_s, C_t are input-dependent projections, V_s is the value vector, and $\gamma \in (0, 1)$ is a per-head decay scalar. In our implementation γ is frozen (not trained) — when trainable, it drifted toward 1.0 (no decay), causing its $O(T^2)$ gradient to dominate the global gradient clip. This is equivalent to attention with a fixed exponential decay mask $D[t, s] = \gamma^{t-s}$ in place of softmax normalization.

The key property for our purposes is that the retention layer maintains a **fixed-size state** $\mathbf{h}_t$ that is updated at each position:

$$\mathbf{h}_t = \gamma\, \mathbf{h}_{t-1} + B_t^\top V_t \tag{2}$$

This state is **compressed**: it has a fixed dimension regardless of sequence length, and it is **input-dependent**: the projections B_t, V_t are computed from the current input. This is functionally analogous to the J-SPACE—a small, privileged state that carries task-relevant information forward through the computation. The decay γ provides a natural forgetting mechanism: old information fades unless actively reinforced by new inputs, preventing unbounded state growth.

The retention formulation is pure matrix multiplications (no selective scan or causal conv1d), which makes it well-suited to the Tenstorrent Blackhole hardware. The backward pass (gradient of the loss w.r.t. γ) is computed via a row-sum/col-sum reduction of the $T \times T$ decay matrix, on-device.

3 The Retention Coincidence

The central architectural observation behind Jasper is what we call the *retention coincidence*: the retention layer's compressed state already maintains a representation with the functional shape of the J-SPACE, without any engineering. Consider the parallels:

Property	**J-space (Anthropic)**	**Retention State**
Fixed size	∼25 concepts, <10% of activation variance	$d_{\text{state}} = 64$ per head, regardless of sequence length
Selective	Broadcasted more widely than other representations	Input-dependent projections control write/forget
Persistent	Carries intermediate reasoning steps across layers	Carries information across the sequence
Causally responsible	Ablation destroys multi-step reasoning	Removing state connections breaks long-range dependencies
Emerges in training	Not designed, emerged in Claude	Not designed for reasoning, emerged from language modeling

Table 1: Functional parallel between the J-SPACE and the retention state.

This parallel is not merely metaphorical. When we build a hybrid model with retention layers and an explicit workspace module, the retention state and the workspace slots *reinforce each other*: the retention state provides a natural substrate for the workspace to read from and write to, and the workspace provides a structured, interpretable interface to the retention layer's otherwise opaque state. In a pure transformer, the workspace must carve out its role from the residual stream against the competing pressure of attention's global mixing. In a hybrid retention model, the workspace aligns with an existing architectural feature.

The coincidence becomes powerful when combined with a recurrent core. In a recurrent transformer (Geiping et al., 2025), each iteration processes the full sequence with attention, growing the KV cache. In a recurrent retention model, each iteration updates the retention state in-place—memory is flat in both sequence length *and* loop count. The workspace state, carried between iterations, provides a stable anchor for iterative refinement without any memory growth.

4 Architecture

4.1 Overview

The Jasper architecture combines four components:

1. **Retention layers** for sequence processing with flat memory. The retention layer uses decayed linear attention with a frozen per-head decay scalar γ, maintaining a fixed-size compressed state while being pure matmuls.
2. **Attention layers** at selected positions for exact retrieval capability.
3. **Workspace module** — a perceiver-style cross-attention block with learned slot vectors, QK normalization, and ReZero gates.
4. **Recurrent core** — a range of layers that are iterated K times, with the workspace state carried between iterations. The iteration outputs are combined via learned attention residuals rather than a fixed blend.

4.2 Workspace Module

The workspace module is an explicit implementation of the J-SPACE concept. It maintains $m = 16$ learned slot vectors $\mathbf{S} \in \mathbb{R}^{m \times d}$, where $d = 384$ is the model dimension. At each

application, it performs two phases:

QK normalization. Both cross-attention passes L2-normalize the query and key vectors along the per-head dimension before computing attention scores, with a learnable per-pass scale parameter s initialized to $1/\sqrt{d_h}$:

$$\hat{\mathbf{Q}} = \frac{\mathbf{Q}}{\|\mathbf{Q}\|_{d_h}}, \quad \hat{\mathbf{K}} = \frac{\mathbf{K}}{\|\mathbf{K}\|_{d_h}}, \quad \mathbf{A} = \text{softmax}\Big(s \cdot \hat{\mathbf{Q}}\,\hat{\mathbf{K}}^\top\Big) \tag{3}$$

This bounds attention logits to $[-s, +s]$ regardless of weight matrix magnitudes, eliminating the entropy collapse and ill-conditioned $\mathbf{QK}^\top$ that caused training divergence in early experiments (condition numbers of 40,000–112,000 were observed). QK normalization is now standard in modern frontier models (OLMo 2, Gemma 3, Qwen 3) (Henry et al., 2020) and was independently validated by Kimi K3's KDA attention (Moonshot AI, 2026). It replaces an earlier spectral norm cap on the workspace weight matrices, which addressed the same root cause (unbounded logits) less directly.

Read phase (sequence → workspace). Slots attend over the hidden states $\mathbf{X} \in \mathbb{R}^{T \times d}$:

$$\mathbf{Q}_s = \mathbf{S}\,W_Q, \quad \mathbf{K}_x = \mathbf{X}\,W_K, \quad \mathbf{V}_x = \mathbf{X}\,W_V \tag{4}$$

$$\mathbf{A}_{\text{read}} = \text{softmax}\Big(s_r \cdot \hat{\mathbf{Q}}_s\,\hat{\mathbf{K}}_x^\top\Big) \tag{5}$$

$$\mathbf{S}' = \text{SlotNorm}(\gamma_s \cdot \mathbf{S} + g_r \cdot \mathbf{A}_{\text{read}}\mathbf{V}_x\,W_O) \tag{6}$$

where g_r is a ReZero gate (scalar, initialized to 0) and γ_s is a learned slot decay scalar (initialized to 1.0). The ReZero gate means the workspace starts as a true no-op ($g_r = 0$) and grows linearly through gradient descent, giving the backbone time to learn a good representation before the workspace contributes. This avoids the gradient saturation problem of sigmoid gates (which we used in earlier experiments and found slow to adapt). The slot decay provides a restoring force: old information fades unless actively reinforced, making the slot update contractive.

Write phase (workspace → sequence). Hidden states attend over the updated slots:

$$\mathbf{Q}_x = \mathbf{X}\,W'_Q, \quad \mathbf{K}_s = \mathbf{S}'\,W'_K, \quad \mathbf{V}_s = \mathbf{S}'\,W'_V \tag{7}$$

$$\mathbf{A}_{\text{write}} = \text{softmax}\Big(s_w \cdot \hat{\mathbf{Q}}_x\,\hat{\mathbf{K}}_s^\top\Big) \tag{8}$$

$$\mathbf{X}' = \text{RMSNorm}(\mathbf{X} + g_w \cdot \mathbf{A}_{\text{write}}\mathbf{V}_s\,W'_O) \tag{9}$$

where g_w is a ReZero gate (scalar, initialized to 0) and s_w is the write-pass QK scale. The slot state $\mathbf{S}'$ is persistent: it is carried forward to the next workspace application and, in the recurrent core, across loop iterations. SlotNorm (RMSNorm applied to the slot dimension) prevents unbounded growth of slot activations across iterations.

Causal masking. Both cross-attention passes use causal masking (Perceiver-IO style (Jaegle et al., 2022)) to prevent future-token leakage in autoregressive settings. Without causal masking, the workspace's bidirectional cross-attention would allow the model to read future tokens (including the answer) through the slots, creating a shortcut that bypasses reasoning entirely. The read pass restricts slot i to positions $[0, (i{+}1)\lfloor T/m \rfloor - 1]$, giving each slot a growing receptive field. The write pass restricts position t to slots $[0, (t{+}1)\lfloor m/T \rfloor]$, ensuring each position only reads from slots whose receptive field includes positions $\leq t$. This preserves the workspace's memory function while respecting causality.

Slot parameter normalization. The learned slot parameters $\mathbf{S}$ are subject to a positive feedback loop: large slots produce sharp attention distributions, which produce large gradients, which cause the slots to grow further. To break this loop, the slot parameters are normalized to unit RMS after each optimizer step:

$$\mathbf{S} \leftarrow \frac{\mathbf{S}}{\mathrm{RMS}(\mathbf{S})}, \quad \mathrm{RMS}(\mathbf{S}) = \sqrt{\frac{1}{md}\sum_{i,j} S_{ij}^2} \tag{10}$$

This is a parameter constraint (analogous to weight clipping in GANs) rather than a learned normalization. It allows the *direction* of each slot vector to change freely while constraining its magnitude, preventing the runaway growth that causes gradient explosion.

Backbone spectral normalization. The backbone attention and retention weight matrices (QKV and output projections) are spectrally normalized to a cap of $C_{\mathrm{bb}} = 2.0$ after each optimizer step, using the same power-iteration technique as Miyato et al. (2018). This prevents unbounded weight growth in the backbone while preserving sufficient capacity. The workspace weights are not spectrally normalized — QK normalization handles that path more directly.

Design rationale. The workspace is deliberately small ($m = 16$ slots, ∼2M parameters total) to mirror the compactness of the J-SPACE. The two-phase design (read then write) mirrors the global workspace theory's broadcast cycle: information is first consolidated into the workspace, then broadcast back to the processing stream.

4.3 Recurrent Core

Layers c_{start} through $c_{\mathrm{end}} - 1$ (by default, layers 6–9) form the recurrent core. These layers are applied K times in sequence, with the workspace state carried between iterations:

$$\mathbf{x}^{(k+1)}, \mathbf{S}^{(k+1)} = \mathrm{CoreLayers}\left(\mathbf{x}^{(k)}, \mathbf{S}^{(k)}\right), \quad k = 0, \ldots, K-1 \tag{11}$$

During training, K is sampled uniformly from $\{1, \ldots, K_{\max}\}$ for each batch. For hardware compilation (e.g., on Tenstorrent accelerators), the loop is always unrolled to $K_{\max}$, with iterations beyond the sampled K masked out. The computation graph is static, allowing variable-depth training without recompilation.

Attention residuals. The final hidden state is computed not by a fixed linear blend of iteration outputs, but by *learned softmax attention over all iteration outputs* (including the pre-core input $\mathbf{x}^{(0)}$). This technique, inspired by Kimi K3's Attention Residuals (Moonshot AI, 2026) and ReZero (Bachlechner et al., 2020), stores all $K + 1$ iteration outputs and computes:

$$\mathrm{score}_k = \left(\sum_d \mathbf{x}_d^{(k)} \cdot \mathbf{w}_d\right) \cdot s, \quad k = 0, \ldots, K \tag{12}$$

$$\boldsymbol{\alpha} = \mathrm{softmax}([\mathrm{score}_0, \ldots, \mathrm{score}_K]) \tag{13}$$

$$\mathbf{x}_{\mathrm{final}} = \sum_{k=0}^{K} \alpha_k \cdot \mathbf{x}^{(k)} \tag{14}$$

where $\mathbf{w} \in \mathbb{R}^d$ is a learned query vector (not input-dependent) and s is a learnable scalar (initialized to $1/\sqrt{d}$). This gives three key properties:

- **Bounded output magnitude:** softmax normalizes weights to sum to 1, so $|\mathbf{x}_{\mathrm{final}}| \leq \max_k |\mathbf{x}^{(k)}|$ — no amplification across iterations.

- **Consistent gradient magnitude:** each iteration receives gradient $\alpha_k \cdot \nabla_{\mathbf{x}_{\text{final}}}$ directly from the attention, not through a chain of blend operations. The gradient is bounded by the softmax weights.
- **Learned selection:** the model learns which iterations' outputs to weight more, rather than using a fixed schedule.

This design was motivated by repeated divergence in earlier experiments with a fixed blend ($\mathbf{x} = \alpha \cdot \mathbf{x}_{\text{new}} + (1-\alpha) \cdot \mathbf{x}$). The fixed blend creates a multiplicative gradient path: each iteration's gradient flows back through all subsequent iterations' blend operations, amplifying by $(1-\alpha)$ per iteration. With $K = 6$ and $\alpha = 1/\sqrt{K}$, the workspace's cross-attention amplification compounds on top of this chain, causing gradient explosion at step ∼1000 in our experiments. The attention residual replaces the chain with direct (bounded) gradient paths.

Slot chain gradient scaling. While the attention residual bounds the $\mathbf{x}$-path gradient, the workspace slot state $\mathbf{S}$ chains multiplicatively across iterations: $\mathbf{S}^{(k+1)}$ depends on $\mathbf{S}^{(k)}$, and the gradient $\nabla_{\mathbf{S}^{(k)}}$ receives chained contributions from all subsequent iterations. In the backward pass, this chained slot gradient is scaled by $1/K$ per iteration, giving a per-iteration gain of $A/K^2 + 1 - 1/\sqrt{K}$ (where A is the workspace's backward amplification factor), which is ≤ 1 for $A \leq \sqrt{K}$. This is more conservative than the $1/\sqrt{K}$ scaling used for additive residual streams, but necessary because the slot chain is multiplicative rather than additive.

At inference time, K can be chosen freely—this is the test-time compute scaling mechanism. Harder problems can be allocated more iterations, directly trading compute for accuracy.

4.4 Hybrid Backbone

The full model uses 13–14 layers total, with attention inserted at positions 5 and 10. The attention layers provide exact retrieval capability—a known weakness of pure retention models (Arora et al., 2024)—while the retention layers provide efficient sequence processing with flat memory. The hybrid ratio (∼85% retention, ∼15% attention) follows the empirical finding that a minority of attention layers recovers most of the retrieval gap while preserving the throughput and memory advantages.

5 Comparison to Thinking Tokens

The dominant approach to scaling reasoning in language models is chain-of-thought (CoT) prompting: the model generates a sequence of intermediate "thinking tokens" before producing its final answer. This approach, while effective, has several structural inefficiencies that the workspace-recurrent architecture addresses directly.

Property	**Thinking Tokens (CoT)**	**Workspace Recurrence (Jasper)**
Reasoning medium	Token space (discrete, serial)	Latent space (continuous, parallel)
Memory growth	Linear in reasoning length (KV cache)	Flat (fixed workspace + retention state)
Tokens generated	One per reasoning step	Zero—reasoning is internal
Legibility	Fully legible (readable text)	Partially legible (workspace probes)
Compute control	Number of generated tokens	Number of loop iterations K
Training data	Requires CoT supervision or RL	Standard next-token prediction + deep supervision
Expressive range	Limited to verbalizable reasoning	Can capture non-verbal reasoning patterns
Deployment cost	High (long generation, large cache)	Low (short output, fixed memory)

Table 2: Comparison of thinking tokens vs. workspace recurrence.

The key distinction is that thinking tokens reason in *token space*—the model must verbalize each intermediate step, serialize it, and re-process it on the next forward pass. This is slow (one forward pass per token), memory-hungry (the KV cache grows with each token), and constrains reasoning to what can be expressed in language. As Geiping et al. (2025) note, "a substantial amount of thought happens through complex, recurrent firing patterns in the brain, before the first word of an answer is uttered."

Workspace recurrence reasons in *latent space*—the workspace state is updated in-place through matrix operations, without generating any tokens. This is faster (one forward pass per iteration, not per token), memory-flat (the workspace has a fixed size regardless of reasoning depth), and can capture reasoning patterns that are not easily expressed in words. The trade-off is legibility: thinking tokens produce readable traces, while workspace reasoning is only partially legible through probes (see Section 6).

For deployment on constrained devices (web browsers, mobile phones), the memory advantage is decisive. A model that generates 1000 thinking tokens accumulates a KV cache of $\sim$1000 $\times$ d_{model} $\times$ n_{layers} floats. A model that iterates its workspace 100 times uses the same fixed workspace state ($m \times d$ floats) plus the retention state ($d_{\text{state}} \times d$ floats), regardless of K.

6 Experimental Design

6.1 Ablation Ladder

We train three parameter-matched model variants ($\sim$10M parameters each) that form an ablation ladder—each cell adds one component, so the marginal contribution of each is isolated:

Cell	Architecture	What it tests
A	Hybrid (14 retention + 2 attention at positions 5, 10), no workspace	The no-workspace control; isolates the workspace effect
B	Hybrid + workspace (13 retention + 2 attention + 16-slot perceiver workspace)	Does an engineered workspace help without recurrence?
C	Full architecture (hybrid + workspace + layers 6–9 looped K times)	The go/no-go cell; does recurrence + workspace beat the control?

Table 3: The three-cell ablation ladder. All cells target $\sim$10M parameters ($d_{\text{model}} = 384$, 13–14 layers). Cell B removes one retention layer to compensate for the $\sim$2M workspace parameters, keeping cells parameter-matched.

Cell history. The design was refined from an initial four-cell grid to the current three-cell ladder after preliminary experiments showed that the pure-backbone and high-LR variants

plateaued on Task 1 without contributing additional signal. The comparison logic is: **B vs A** isolates the workspace effect (same LR=2×10^{-4}); **C vs B** isolates the recurrent core; **C vs A** is the full R1 test.

6.2 Training Configuration

All cells use AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$) with weight decay 0.1 and gradient clipping at norm 1.0. All cells use peak lr=2×10^{-4} with a linear warmup over 200 steps, followed by a constant peak. The training budget is 10,000 steps per cell. Training runs on a Tenstorrent Quietbox 2 (4× Blackhole chips, bfloat16-native) with all three cells in parallel on separate devices.

Batch configuration. Each step uses micro-batch size 8 with 48 gradient accumulation steps, giving an effective batch size of 384 sequences (∼49K tokens per step). Fresh data is sampled every batch—with generated data there is no train set to overfit.

Early stopping. An EMA-smoothed loss plateau detector stops training if the loss does not improve by a relative 0.1% over 1,000 steps (10% of the training budget). The EMA smoothing factor is $\beta = 0.99$, giving a half-life of ∼69 steps (∼3,300 micro-batches), which is well above the per-step noise floor at this batch size.

Per-parameter LR groups. The workspace parameters (∼2M of ∼10M total) receive 0.25× the base learning rate (5×10^{-5} vs 2×10^{-4} for the backbone). This is motivated by the observation that the workspace creates a sharper loss landscape: at equal LR, workspace gradients dominate and cause instability. The backbone LR is kept at 2×10^{-4} to match Cell A (the control), preserving the clean B-vs-A comparison. Gate parameters (read/write gates) receive 1× the base LR — reduced from an earlier 10× after evaluation showed that high gate LR caused premature gate opening before the workspace had learned useful representations, destabilizing the backbone.

Stabilization techniques. The workspace and recurrent core introduce several sources of gradient instability that are not present in the baseline hybrid model. We address each with a targeted normalization:

1. **Per-parameter LR groups** (above): workspace parameters receive 0.25× the base LR, preventing the workspace from dominating the gradient. Gate parameters receive 1× base LR (reduced from 10× after evaluation showed premature opening destabilized the backbone). QK scale and attention residual parameters receive 1.0× base LR (backbone speed).
2. **QK normalization** (Section 4.2): L2-normalizes Q and K per head before attention, bounding logits regardless of weight magnitudes. This eliminates the entropy collapse (condition numbers 40,000–112,000) that caused divergence in early experiments.
3. **ReZero gates** (Section 4.2): scalar gates initialized to 0.0 (true identity at init), no sigmoid. The workspace starts as a no-op and grows through gradient descent, avoiding the sigmoid's gradient saturation.
4. **Gate freeze** (new): during the first N_{freeze} steps (3,000 for synthetic, 600 for text), workspace gates are held at exactly 0 after each optimizer step via `freeze_workspace_gates()`. This makes the workspace a true identity (no-op) during the freeze, so the backbone trains as if the workspace doesn't exist. After the freeze, gates learn freely at 1× LR. This was added after evaluation showed that even with ReZero initialization, noisy gradients at high gate LR caused gates to drift open before the workspace had learned useful representations,

destabilizing the backbone. The freeze ensures a clean B-vs-A comparison: B trains as exactly A during freeze, then the workspace opens only if gradient descent finds it useful.

5. **Attention residual core** (Section 4.3): replaces the fixed blend with learned softmax attention over iteration outputs, bounding output magnitude and providing direct (non-chained) gradient paths.
6. **Slot chain $1/K$ scaling** (Section 4.3): scales the chained slot gradient by $1/K$ per iteration in the backward pass, preventing multiplicative gradient amplification across the K-iteration slot chain.
7. **Slot parameter normalization** (Section 4.2): constrains the learned slot parameters to unit RMS after each optimizer step, breaking the positive feedback loop where large slots $\rightarrow$ sharp attention $\rightarrow$ large gradients $\rightarrow$ larger slots.
8. **Backbone spectral normalization** (Section 4.2): caps the spectral norm of backbone weight matrices at $C_{\text{bb}} = 2.0$ after each optimizer step.
9. **Retention γ freezing**: the retention layer's decay scalar γ is frozen (not trained). When γ was trainable, it drifted to 1.0 (no decay), causing its $O(T^2)$ gradient to dominate the global gradient clip and starve all other parameters. Freezing eliminates this gradient scale mismatch entirely.
10. **Component-wise gradient clipping**: workspace parameters (names starting with `ws_`) are clipped at norm 0.5, backbone parameters at norm 1.0, with each group's norm computed and clipped independently. This prevents workspace gradient spikes from dominating the global clip and starving the backbone of learning signal (Yang et al., 2022).
11. **Restore-on-spike**: if the pre-clip gradient norm exceeds 5000, the model and optimizer state are restored from the last checkpoint rather than skipping the step. Skip-on-spike freezes the model in the bad state that caused the spike — it can never recover. Restore gives a fresh start from a known-good state (at most 100 steps old).

Each technique targets a distinct mechanism. The first six are architectural (they change the forward or backward computation); the last five are training-time (they constrain parameters or gradients). All are deterministic and can be disabled independently for ablation.

Custom fused kernels. The Tenstorrent implementation uses three custom tt-metal kernels to reduce op dispatch overhead and eliminate intermediate DRAM writes: (1) fused RoPE (full rotation in a single kernel pass, using a split-RoPE optimization that avoids sub-tile concat/slice when $d_{\text{head}}/2$ is not tile-aligned), (2) fused scale+decay (scores $= \text{qk} \cdot \text{scale} \cdot D_{\text{decay}}$ in one pass), and (3) fused gate backward (computing grad_out_flat and grad_g in a single kernel pass). The retention layer runs at 2.9 ms per forward+backward pass at $d_{\text{model}} = 384$, $n_{\text{heads}} = 4$, $T = 128$, $B = 8$.

Attention regularizers (implemented, disabled). To counter workspace degeneracy we implemented an attention entropy penalty and a slot diversity penalty (`ws_entropy_weight`, `ws_diversity_weight`). Both are **disabled**, at weight 0. The entropy penalty is disabled because entropy measures *peakedness*, not *input-dependence*: the cheapest way to minimize attention entropy is a fixed one-hot routing that ignores the input entirely, which is precisely the degenerate solution the regularizer was intended to prevent. The code paths are retained for reproducibility. Selectivity is now measured, not optimized: see the input-dependence diagnostic below.

Selectivity and decoding diagnostics. Two measurements replace the discarded entropy objective. **Input-dependence**: probe the model with $N = 16$ distinct inputs of identical token length and compute the mean total variation distance between each input's attention distribution and the across-input mean, holding the position index fixed, plus the fraction of positions whose top-1 target is identical across all probes. A fixed routing scores TV $= 0$ and

100% agreement no matter how peaked it is, so this cannot be gamed by sharpening. Reported per path (read and write) because a functioning read path can otherwise mask a degenerate write path. **Teacher-forced answer accuracy**: evaluate answer tokens given the gold prefix, alongside free generation, to separate reasoning failure from exposure bias.

6.3 Synthetic Tasks

All tasks are character-level, generated on-the-fly (infinite data, no overfitting), and automatically verifiable. Each has a **depth knob** that controls how many reasoning steps a correct answer requires.

Task 1: Chained assignment arithmetic (multi-hop composition). Variables are defined in terms of previous variables, all arithmetic mod 97:

```
a=7;b=a*3+2;c=b-a;d=c*2;?d;
```

The model must follow a chain of k dependencies. Distractor variables are inserted at random positions. This is the core multi-hop reasoning task.

Task 2: Permutation tracking (state-tracking). n items undergo k swap operations; the model must report one item's final position:

```
n=6;2,5;0,3;5,1;?3;
```

State-tracking is a documented weakness of linear attention models (Arora et al., 2024); this task tests whether the workspace compensates for the architecture's known gap.

Task 3: Single-hop recall (control). Shallow lookup with distractors:

```
a=5;b=12;c=8;d=3;e=15;?c;
```

This is the control for selective ablation: the J-SPACE signature requires that killing the workspace leaves this intact while Tasks 1–2 collapse.

Training uses depths 2–8; evaluation extends to 2–16, measuring extrapolation beyond the training distribution.

6.4 Pre-Registered Results

Four measurements constitute proof, in ascending order of importance:

R1 — Capability:	Cell C beats Cell A by $\geq$10 accuracy points on Tasks 1–2 at depths 10–16 (beyond training range).
R2 — Compute scaling:	Accuracy on deep problems increases monotonically with K; harder depths need higher K to saturate.
R3 — Workspace decodability:	Linear probes on workspace slots can decode intermediate variable values, with decodability rising across loop iterations. This is the J-lens analogue.
R4 — Selective ablation:	Replacing workspace slots with their training-set mean causes Tasks 1–2 to collapse ($\geq$30pt drop) while Task 3 barely moves ($\leq$5pt drop)—mirroring Anthropic's J-SPACE finding.

R3 and R4 are what make this a *J-SPACE proof* rather than just another architecture ablation. R4 in particular tests the causal claim: the workspace is not merely correlated with reasoning performance, it is *causally responsible* for it.

7 Preliminary Results

The current training run (August 2026) uses the retention-based backbone on Tenstorrent Blackhole hardware with the three-cell ablation ladder (A, B, C). Cell A has completed training (10,000 steps); Cells B and C were evaluated at their pre-restart checkpoints, then restarted from scratch with the gate freeze fix. A text training run on a mixed reasoning corpus (tiny challenges) is training in parallel.

Evaluation of pre-restart checkpoints. After fixing the evaluator (TT-Metal's batch-size-1 matmul limitation was handled by padding to batch 4), all three cells were evaluated on the synthetic tasks before the v2 restart:

Cell	Step	Task 1	Task 2	Task 3	Overall	Depth 2	Depth 8
A	9600	0.0%	97.5%	100.0%	65.8%	66.7%	63.3%
B	8800	0.0%	42.5%	5.0%	15.8%	30.0%	3.3%
C	4100	2.5%	40.0%	40.0%	27.5%	43.3%	20.0%

Table 4: Evaluation of pre-restart checkpoints. Task 1 = chained assignment arithmetic (multi-hop), Task 2 = permutation tracking (state-tracking), Task 3 = single-hop recall (control). None of the cells learned Task 1. Cell B (workspace, no recurrence) performed *worse* than Cell A (control), indicating the workspace actively destabilized the backbone.

Key findings:

- **None of the cells learned Task 1** (chained assignment arithmetic), even Cell A at step 9600. The multi-hop reasoning task is beyond what a 10M retention model can learn from synthetic character-level data at this scale.
- **Cell B is worse than Cell A** on Tasks 2 and 3. The workspace actively hurt the backbone: gradient norms were 250–2100 vs A's ∼8, and 12% of training steps were wasted on restore/skip events. The 3× gate LR caused gates to drift open via noisy gradients before the workspace had learned useful representations.
- **Cell C is better than Cell B** (27.5% vs 15.8% overall) with stable gradients (0 restores), but still worse than Cell A on Tasks 2/3. The recurrent core appears to mediate the workspace's gradient amplification, but not enough to overcome the backbone destabilization.
- **Teacher-forced accuracy matches free-generation accuracy** for all cells, confirming the failures are reasoning failures, not decoding artifacts.

Gate freeze and v2 restart. The evaluation showed that the workspace was actively harming the backbone rather than merely failing to help. The root cause was premature gate opening: even with ReZero initialization (gates start at 0), the 3–10× gate LR caused gates to drift open due to noisy gradients before the workspace had learned useful representations.

The fix adds a **gate freeze period**: for the first N_{freeze} steps, workspace gates are held at exactly 0 after each optimizer step. The workspace is a true no-op (identity) during freeze, so the backbone trains as if the workspace doesn't exist — Cell B becomes exactly Cell A. After the freeze, gates learn freely at 1× LR (reduced from 3–10×). This ensures a clean B-vs-A comparison: if B matches A during freeze and the workspace opens but doesn't help after unfreeze, that supports the conclusion that workspace alone is insufficient.

Cells B and C were restarted from scratch with gate freeze (3,000 steps) and 1× gate LR. At initialization, both show gradient norms ∼8.8 matching Cell A, confirming the workspace is a true no-op during freeze.

Cell A: completed. Cell A (backbone-only control, 10.08M parameters) completed training at step 9,999 with final loss 0.83. It mastered Tasks 2 and 3 (97.5% and 100%) but scored 0% on Task 1, confirming that the multi-hop reasoning task is beyond the backbone's capacity at this scale — the workspace and recurrent core must provide the missing capability for the architecture to succeed.

Cells B v2 and C v2: in progress. Both cells are training from scratch with gate freeze. As of this writing, both are in the freeze period (gates at 0.0, gradient norms ∼8.8, matching Cell A's trajectory). The critical observation point is step 3,000 when gates unfreeze: if B v2 matches A on Tasks 2/3 at that point, the clean baseline is established. If the workspace then opens and B v2 does not improve, that supports the "workspace alone is insufficient" hypothesis. Cell C v2 tests whether the recurrent core provides the missing ingredient.

Text training: tiny challenges corpus. In parallel with the synthetic POC, a text training run is testing the architecture on a 2M-example mixed reasoning corpus ("tiny challenges"), inspired by Enigmata-style synthetic reasoning training. The corpus mixes narrative logic puzzles (40%), BrainBashers-style attribute/elimination puzzles (55%), and bAbI QA (5%), for 78.7M training tokens. The shift from TinyStories to tiny challenges reflects the finding that the synthetic POC Task 1 was not learnable at 10M parameters — the text corpus tests whether framing multi-hop reasoning as text continuation is more learnable.

The text model (29.8M parameters including 19.3M embedding) trains with gate freeze for 600 steps, $K_{\max} = 6$, seq_len=512, effective batch 384, at ∼112 s/step (∼62 hours for 2,000 steps). Initial loss is 10.92 ($\approx \log 50257$, confirming correct random init), gradient norm 1.5, gates at 0.0.

History of gradient instability in Cell C. Cell C required five rounds of fixes before achieving stable training, each addressing a distinct gradient amplification mechanism:

1. **QK normalization** (step ∼700–900 divergence): The workspace's $\mathbf{QK}^\top$ had condition numbers of 40,000–112,000 (entropy collapse). When gates opened to ∼0.20, cross-attention gradient paths amplified backbone gradients multiplicatively. QK normalization bounds logits regardless of weight magnitudes.
2. **ReZero gates** (same period): Sigmoid gates initialized at -2 ($\sigma(-2) \approx 0.12$) had saturating gradients that made them slow to adapt. ReZero gates (init 0, no sigmoid) start at true identity and grow linearly.
3. **Slot chain $1/K$ scaling** (step ∼750–1150 divergence): The slot state chains multiplicatively across iterations, and the workspace's gradient amplification compounds on this chain. Scaling the chained slot gradient by $1/K$ per iteration gives a per-iteration gain ≤ 1 for $A \leq \sqrt{K}$.
4. **Attention residual core** (step ∼1000 divergence): The fixed blend $\mathbf{x} = \alpha \cdot \mathbf{x}_{\text{new}} + (1-\alpha) \cdot \mathbf{x}$ creates a multiplicative gradient path. Replacing it with learned softmax attention over iteration outputs provides bounded, direct gradient paths.
5. **AR chain gradient scaling** (step ∼1550 divergence): Even with the attention residual, the gradient from each iteration flows through the shared core layers at full magnitude and chains to the previous iteration. Scaling this chained gradient by $1/K$ (matching the slot chain scaling) gives a per-iteration chain gain of A/K, stable for $A < K$.

A sixth fix — **gate freeze** — was added after evaluation showed that even with all gradient instability resolved, premature gate opening at high gate LR destabilized the backbone before the workspace could learn useful representations.

Backward pass verification. The workspace backward pass has been verified against PyTorch autograd on CPU. All gradients—input gradients, weight gradients (8 matrices), slot gradients,

norm weights, gate scalars, QK scale parameters, and attention residual parameters (query vector and scale)—match to within float32 precision ($< 10^{-4}$ relative error). The retention layer backward (including the decay matrix gradient) is verified separately via a dedicated parity test against a float64 reference.

8 Discussion

8.1 Why This Might Work

The J-SPACE finding tells us that reasoning in large models is carried by a small, privileged subspace. The TRM finding tells us that iteration through a small network can substitute for parameter count. The retention coincidence tells us that the retention layer's compressed state already has the functional shape of the J-SPACE. Jasper combines all three: the workspace provides the privileged subspace, the recurrent core provides the iteration, and the hybrid retention backbone provides the substrate.

If this works, the implication is that we can build reasoning models that are:

- **Small enough for the browser:** ∼1B parameters, flat memory in both sequence length and loop count.
- **Controllable at inference:** K can be adjusted per problem, trading compute for accuracy.
- **Interpretable:** workspace probes (R3) provide a window into the model's reasoning, unlike opaque latent reasoning in recurrent transformers.
- **Verifiable:** selective ablation (R4) provides a causal test of whether the workspace is doing the reasoning, not just correlated with it.

8.2 Why It Might Not Work

Synthetic-task wins at 30M parameters have a real history of not transferring to language at scale. The tasks we use (arithmetic chains, permutation tracking) are structurally simple and may not capture the full complexity of language reasoning. The workspace may help on these tasks without generalizing to the kind of reasoning that matters for real applications.

The gradient stabilization techniques consumed most of our engineering effort, and we should be clear that little of what resolved them was novel. QK normalization is standard since OLMo 2; ReZero gates are from Bachlechner et al. (2020); component-wise gradient clipping is from Yang et al. (2022); a parameter inside a positive feedback loop needs its magnitude constrained, so slot normalization is routine; an additive module should begin at identity, standard since LoRA; and a recurrent state with no forget term will not stay bounded, which LSTMs settled in 1997. The attention residual core is the one component we adopted from concurrent work (Kimi K3) rather than deriving ourselves, and it was the fix that finally broke the gradient amplification chain in Cell C. These properties belong in the initial design, and we present them in Section 6.2 as design requirements derived from the architecture rather than as findings.

What was not obvious, and what we got wrong, is the measurement. We initially adopted attention entropy as a proxy for selectivity and optimized it directly. This is a measurement error: entropy scores peakedness rather than input-dependence, and the cheapest way to minimize entropy is a fixed routing that ignores the input—precisely the degenerate solution the regularizer was intended to prevent. The general lesson—that a proxy admitting a degenerate optimum will find it, and that "sharp" must be distinguished from "informative"—transfers beyond this architecture, and it is the part of this work we would defend.

The more substantive question is whether the workspace provides enough *computational leverage* to matter. The pre-restart evaluation gave a discouraging signal: Cell B (workspace, no recurrence) performed *worse* than Cell A (control) on Tasks 2 and 3, with 12% of training steps wasted on gradient spike recovery. The workspace was actively destabilizing the backbone

rather than helping it. However, this was with premature gate opening at high gate LR — the v2 runs with gate freeze will provide a fairer test. If B v2 matches A during freeze and the workspace opens but doesn't help after unfreeze, that cleanly establishes that workspace alone is insufficient. The R1–R4 criteria (Section 6) are designed to distinguish these explanations.

Two possibilities remain open. The first is that our experimental design tests the wrong thing: Task 1 requires iteration *and* memory, and Cell B supplies memory without iteration, so B-versus-A may be architecturally incapable of showing a benefit regardless of workspace quality. On this reading the informative comparison is C-versus-B, and we have under-prioritized C because it is slower. The second is that the workspace as designed is simply not the right mechanism. It requires eight weight matrices, two ReZero gates, a decay term, QK normalization, slot normalization, gate freeze, and four learning-rate groups to train at all; that quantity of scaffolding is itself evidence against the design, since the architectures that won in practice—transformers included—won by being simple and trainable rather than by being optimal. Accordingly we have set an advance stopping rule: if Cell C v2 shows Task 1 $\leq 10\%$ by step 5,000 (well past the 3,000-step gate unfreeze) with gates verifiably open and attention verifiably input-dependent, we report the negative result rather than continue to search for one more fix.

Finally, the retention coincidence may be more metaphorical than functional. The retention state and the J-SPACE have similar shapes, but they operate at different levels of abstraction—the retention state is a low-level sequence summary, while the J-SPACE is a high-level conceptual workspace. Whether the workspace module can bridge this gap is an empirical question.

8.3 Decision Framework

The proof-of-concept is designed to produce a clear go/no-go decision before any significant cloud spend:

- **Go:** R1 and R2 pass, and at least one of R3/R4 shows the workspace doing real causal work → fund a 300M language-scale ablation (∼$3–5K).
- **Pivot:** R1 fails but Cell A's efficiency story stands → keep the hybrid backbone, drop the workspace, proceed on hybrid distillation.
- **Kill:** Cell C ≤ Cell A everywhere → the novel architecture track is dead; fall back to sampled-attempts-plus-verifier on a standard model.

The entire proof-of-concept costs ∼$20–30 of electricity and under a week of compute. This buys the right to spend $5K intelligently, not the conclusion itself.

9 Related Work

Global workspace in LLMs. Anthropic (2026) discovered the J-SPACE in Claude Opus 4.6 using the Jacobian lens, showing that a small set of verbalizable representations functions as a global workspace (Baars, 2005; Dehaene, 2014). Our workspace module is an explicit engineering of this concept.

Iterative and recurrent reasoning. Jolicoeur-Martineau (2025) showed that tiny recursive networks (7M parameters) can outperform much larger models on ARC-AGI through iterative refinement. Geiping et al. (2025) scaled recurrent-depth transformers to 3.5B parameters, demonstrating test-time compute scaling in latent space. Gehring et al. (2024) explored relaxed recursive transformers with shared parameters. Our recurrent core follows the TRM paradigm but uses a hybrid retention/attention backbone and an explicit workspace state. The attention residual mechanism in our recurrent core was inspired by Kimi K3 (Moonshot AI, 2026), which uses softmax attention over layer outputs instead of standard residual connections to achieve bounded output magnitude and consistent gradient flow across depth.

Retention and hybrid models. Sun et al. (2024) introduced the Retentive Network, replacing softmax attention with decayed linear attention that maintains a fixed-size recurrent state. Hybrid models that combine recurrent-state layers with attention have been shown to match transformer-based R1-distilled models on AIME/MATH while generating 3× faster, and NVIDIA's Nemotron line has repeatedly shown hybrid models matching or beating same-size transformers on long reasoning traces. Our architecture follows the hybrid pattern but adds the workspace and recurrent core.

Test-time compute scaling. The broader literature on scaling test-time compute—from self-consistency (Wang et al., 2023) to process reward models (Lightman et al., 2023) to OpenAI's o1/o3 models—operates in token space. Our approach scales test-time compute in latent space, which is more memory-efficient but less legible.

10 Conclusion

We have presented Jasper, an architecture that combines three recent insights—the J-SPACE as a compact reasoning workspace, iterative refinement as a substitute for parameter count, and the retention layer's compressed state as a native substrate for the workspace—into a single system. The architecture reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and provides interpretable and causally testable reasoning through workspace probes and selective ablation. The retention backbone (decayed linear attention) preserves the key properties of a fixed-size, input-dependent state while being simple and fast on the target hardware.

The proof-of-concept experiment is designed to produce a clear verdict at minimal cost: if the workspace causally carries multi-step reasoning (R4) and the recurrent core provides test-time compute scaling (R2), the architecture warrants scaling to language-scale experiments. If not, the hybrid retention-attention backbone still stands as an efficient baseline for in-browser deployment.

The broader claim is that reasoning in language models need not be tied to token generation. The brain reasons through recurrent firing patterns before articulating a thought; perhaps language models should too.

References

Wes Gurnee and Jack Lindsey et al. Verbalizable representations form a global workspace in language models. *arXiv preprint arXiv:2607.15495*, 2026.

Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Alexandre Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025. [Samsung SAIT Montreal]

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Bernard J. Baars. Global workspace theory of consciousness: Toward a neuroscience of human experience. *Progress in Brain Research*, 150:45–53, 2005.

Stanislas Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking, 2014.

Simran Arora, Sabhash Rao, and Christopher Ré. Theoretical limitations of self-attention and state space models in reasoning tasks. *arXiv preprint*, 2024.

Jonas Gehring, Kilian Golde, and David Grangier. Relaxed recursive transformers. *arXiv preprint arXiv:2410.10672*, 2024. [Google DeepMind]

Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Hunter Lightman et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. [OpenAI]

Hongyu Wang et al. DeepNet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022. [Microsoft]

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huan Wang, and Caiming Xiong. ReZero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence (UAI)*, 2020.

Alex Henry, Prudhvi Raj Dachuri, and Parikshit Ram. Query-key normalization for transformers. *arXiv preprint arXiv:2010.04245*, 2020.

Kevin Clark, Paul Michel, and Christopher D. Manning. Unified scaling laws for routed language models. In *EMNLP*, 2022.

Moonshot AI. Kimi K3: A scalable mixture-of-experts model with attention residuals. 2026.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Zhangyin Feng, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2024. [Microsoft]

Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, et al. Perceiver IO: A general architecture for structured inputs & outputs. In *International Conference on Learning Representations (ICLR)*, 2022.